
MEMO: Nvidia CUDA programming get started

CUDA Programming Guide

- CPU/GPU をホスト/デバイスと呼び分けている
- CUDA API を通してホストメモリからデバイスメモリにデータをコピーすることができる
- Graphics Processing Clusters(GPCs) は Streaming Multiprocessors(SMs) の集まり
- SM は a local register file と a unified data cache と計算を行なう多数の機能ユニットを持つ
- 統合データキャッシュは共有メモリと L1 キャッシュの実体である
 - 統合データキャッシュ -L1 (共有メモリ, L1 キャッシュ) への割り当ては実行時に設定できる
- カーネル (GPU 上で実行される関数) はスレッドと呼ばれる単位で計算を行う
 - スレッドをまとめたものをスレッドブロックという
 - スレッドブロックをまとめたものをグリッドという
 - * それぞれのまとめ方は 1,2,3 次元
 - * 同じグリッド内のブロックの形 (次元数) は同じ
 - こうしてたくさん名称があるのは、処理したいデータ項目とスレッドを対応付けやすいため (実装の都合の、仮想的な概念か?)
- 同じスレッドブロックに存在するスレッドは同じ SM で実行される
 - パフォーマンス面で言うと、同じ物理的なメモリを共有することが効く
 - communicate や synchronize が有利
 - 逆に、異なるスレッドブロックの間のデータのやり取りはオーバーヘッドが大きいことに注意すべし
- ビット演算の OR ばかりなのになんで atomicOr() が必要なのか
 - 直感的には OR 演算が競合しても順不同だから atomicOr() が必要ないように思えるが、そもそも\=が不可分な 3 つの操作のセットだから
 - read-modify-write と他の read-modify-write のタイミングが競合すると、lost-update が起きてしまう
 - * 2 つの modify のうち、片方はあとからくる write によって上書きされてしまう

```
1 tmp = data[i]; // load
```

```
2 tmp = tmp | mask; // OR
3 data[i] = tmp; // store
```

参考

- Kazuki Fujii, CUDA Programming Guide Part 1
 - <https://zenn.dev/kaz20/articles/1e622ef249d133>
- Nvidia, CUDA Programming Guide
 - <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>

Listing 1 How to build CUDA program

```
1 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/
   vectorAdd$ mkdir build
2 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/
   vectorAdd$ cmake -S ./ -B ./build/ -DCMAKE_CUDA_ARCHITECTURES=86
3 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/
   vectorAdd$ ./build/
4 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/
   vectorAdd$ ./build/vectorAdd
5 Unified Memory: CPU and GPU answers match
```

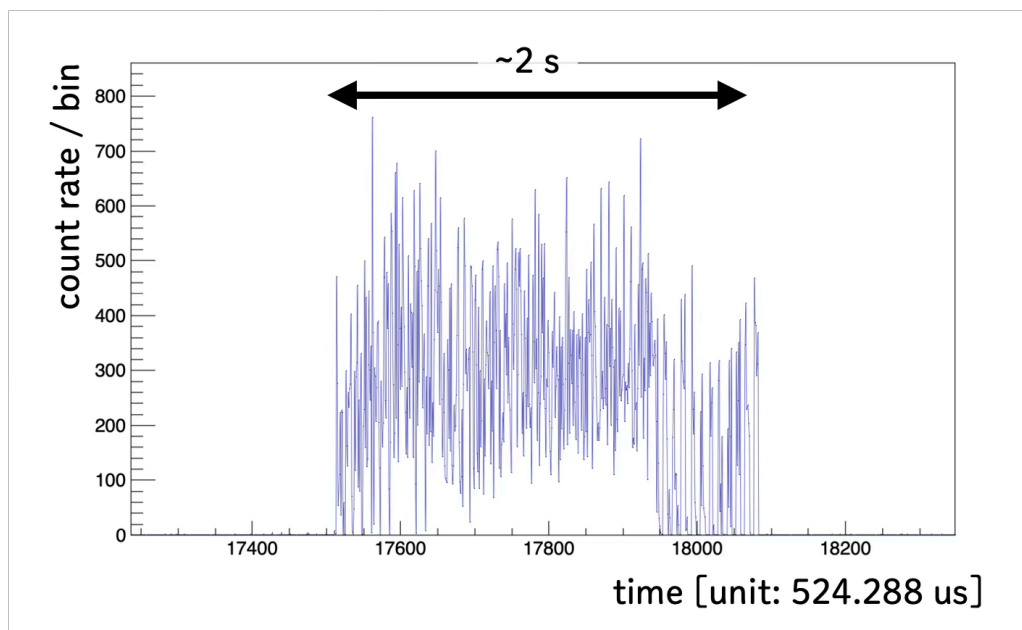


図1 J-PARC ハドロン実験施設に供給されるビームの時間的な構造の例。約2秒の間、加速器施設のメインリングから徐々にビームを取り出す Slow extraction と呼ばれる方式でビームを検出器に照射している。横軸は時間で、縦軸は検出器のカウントレートである。